LLM Router Research

Nov 4, 2026 · Harry

Build three routers, in this order: Arch-Router, Avengers-Pro, CARROT. On our own data, picking the best model per task from the performance table already matches or beats every query-level router. So the first router classifies the request; the next two save cost and latency.

Is our data enough?

For routing on accuracy, yes: six benchmarks have a per-query score that joins across models. For routing on cost and on real traffic, not yet.

Benchmark	Queries scored for all five models	Tokens + latency logged
NegotiationToM	13,826	Luna, Gemini
DocVQA	5,349 (four models; Deepseek is text-only)	none
bbh	4,833	none
AwareBench	4,015	all five
MMLU	3,943	none
EmoBench	400	Luna, Gemini
MultiChallenge, Wonderbread	generated; judge scores still on Quest	all models

Gaps to close before training:

- **Cost.** Prices are not recorded for Qwen, Gemma or Deepseek, and bbh, MMLU and DocVQA logged no token usage.
- **Real requests.** Every row is a benchmark item. Collect 500–2,000 requests shaped like real use, answer them with all five models and grade them with the Haiku 5.5 judge (estimate: $10–40 to generate, ~$5 to judge).
- **Consistency.** MMLU mixes two scorers and two prompt versions. The legacy model behind `deepseek-reasoner` was retired on 2026-07-24, so confirm which model produced the older Deepseek rows.

What can routing gain on our data?

Measured offline: about 1.5 points of macro accuracy, and the performance table already captures it. The five models agree on most items; NegotiationToM and EmoBench are where they differ.

Benchmark	Best single model	Query-level oracle	Models disagree
NegotiationToM	Luna 0.527	0.699	41% of items
EmoBench	Gemma 0.745	0.873	39%

Benchmark	Best single model	Query-level oracle	Models disagree
MMLU	Luna 0.926	0.970	18%
AwareBench	Deepseek 0.961	0.985	17%
DocVQA	Qwen 0.970	0.990	11%
bbh	Gemma 0.971	0.984	11%

Pooled over the five benchmarks all models ran, with every router scored on held-out folds:

Router (macro accuracy, held-out folds)	Score
Always Qwen	0.756
Always Luna	0.793
Always Gemini	0.794
Always Deepseek	0.803
Always Gemma (best single model)	0.811
Query-level kNN, k=10 / 50 / 100	0.819 / 0.820 / 0.821
Query-level logistic regression	0.821
Table: best model per sub-task	0.826
Table: best model per benchmark	0.826
Oracle (ceiling)	0.902

Measured from per-query result files on disk, 2026-11-04 · 5-fold grouped cross-validation · TF-IDF features.

Most of the gap to the 0.902 oracle cannot be routed: Luna's own answers flip on 15–19% of items when it is simply rerun.

Three routers for this project

#	Router	What it does	Why it fits us	Training
1	Arch-Router (Katanemo, 1.5B)	Matches a request to policies we write, one per taxonomy category, and returns the model tied to that policy	The best model per benchmark scored 0.826 vs 0.811 for always-Gemma, and no query-level router did better. Arch-Router makes that pick with no training, and its decisions are readable	None: pretrained
2	Avengers-Pro	Embeds requests, groups them into ~60 clusters, scores each model per cluster as α·accuracy + (1−α)·(1−cost)	Adds cost to the choice: where models tie it picks the cheaper one, and ties are common (models disagree on only 11–18% of items on four of six benchmarks)	Clustering only: minutes on a laptop
3	CARROT (kNN variant)	Predicts each model's chance of being right and its cost for this request, then picks the best trade-off	Cost varies per request, not only per model: Deepseek writes ~150 reasoning tokens per AwareBench row, the others almost none	kNN: minutes; RoBERTa variant ~1 GPU-hour

Considered and not chosen as a first build:

Router	Why not first	Training
RouteLLM	Routes between two models only (strong vs weak); five need a wrapper	Minutes
IRT-Router	Gives interpretable ability scores per model, useful for the write-up rather than for routing gains	Minutes to 1 h
Not Diamond	Closed, at most 10,000 rows per training job; keep it as the commercial baseline	Hosted, minutes to 1 h
GraphRouter, RouterDC	More training for no measured gain over kNN in LLMRouterBench	1–3 GPU-hours
FrugalGPT	A cascade of sequential calls adds latency	~1 GPU-hour
PickLLM	Ignores the request and settles on one model per session	Online
MoA, LLM-Blender, DeePEn, FuseLLM	Ensembles: 2–6 calls per request, or need token probabilities and open weights	—

How to implement

1. **Build one training table first (all three routers use it).** One row per query × model: query text, benchmark, score, tokens, latency, cost. 27,017 queries are ready. Close the gaps listed above first, and add MultiChallenge and Wonderbread once their judge scores come down from Quest.
2. **Arch-Router.** Write one policy per taxonomy category (a name and a one-line description). Map each to its best model from `Final_result.xlsx`, and serve the model from Hugging Face on a Quest GPU. A request with no confident match goes to Gemma.
3. **Avengers-Pro.** Use the authors' code. Embed the queries, cluster with k-means (k ≈ 60), and store each model's accuracy and cost per cluster. Sweep α to choose the cost–accuracy point.

4. **CARROT.** On the same embeddings, a kNN (k ≈ 50–100) predicts each model's P(correct) and cost. Route to the model with the best P − λ·cost, and sweep λ.
5. **Test before adopting.** Score all three on the same held-out folds against always-Gemma and the plain table. Keep a router only if it is more accurate, or as accurate for less cost or latency, by more than run-to-run noise (about 3 points on EmoBench).

How to tell it works

- **Baselines:** always-Gemma (the strongest), always the cheapest model, random, the plain table, and the oracle as the ceiling.
- **Splits:** grouped 5-fold cross-validation (a dialogue or question never on both sides), leave-one-benchmark-out for unseen task types, and the realistic-request set as the final test.
- **Metrics:** macro accuracy with benchmarks weighted equally, cost per 1,000 requests, and median and p90 latency. Sweep α or λ to draw the cost–accuracy curve.
- **Noise floor:** a paired bootstrap per benchmark. For scale, Luna's six EmoBench runs span 0.658–0.688.

Can a small router recognise the request?

Recognising the *kind* of request is easy; predicting *which model gets this one right* is not, at any model size. A TF-IDF + logistic-regression classifier, scored on held-out folds, names the benchmark with 0.998 accuracy and the sub-task (48 classes) with 0.977. Most of its errors are MMLU subjects confused with each other, which route to the same model anyway. Treat 0.998 as an upper bound: benchmark items have distinctive formats, and real requests that mix categories, or fall between mission analysis and goal specification, will be harder. Measure it on the realistic-request set before relying on it.

A wrong category is cheap. Sending a request to the second-best model instead of the best costs 0.7 points (bbh, MMLU), 1.3 (NegotiationToM), 3.0 (AwareBench) and 4.7 (EmoBench). The worst model, Qwen on all five benchmarks, costs 3.4–11.0 points. So add a confidence threshold, and send anything below it to Gemma.

Reproduce

Both scripts read the per-query result files on disk and call no model; on a laptop the first takes about a minute, the second about three:

```
/opt/miniconda3/bin/python3 -I Router_research/router_headroom.py   # headroom, routers, LOBO → headroom.json
/opt/miniconda3/bin/python3 -I Router_research/intent_check.py      # task-type classifier and misroute cost
```

Sources

Arch-Router (from search results, not opened) · Avengers-Pro · CARROT · LLMRouterBench · kNN routers. All accuracy figures for our models were computed offline from our per-query result files.